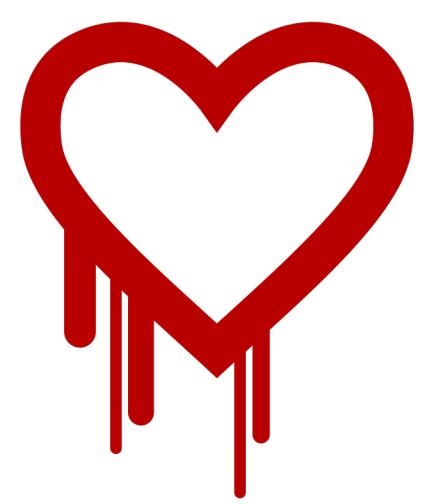




CVE-2014-0160

HEARTHBLEED



SUSCRÍBETE



drovoh4k



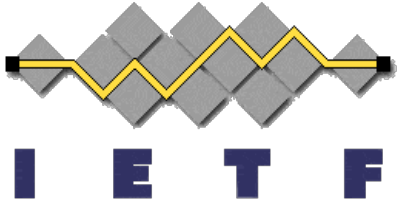
Drovo**hub**

**De dónde viene
el nombre
de HeartBleed**



drovoh4k

Cronología ● Enero 2011



Publica primer borrador para
nueva extensión para TLS

Lo firman tres autores:



Robin Seggelmann



Michael Tuexen



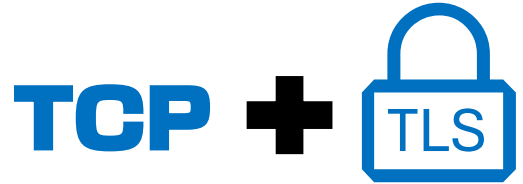
Michael Williams



drovoh4k

Cronología ● Enero 2011

¿Qué problema querían resolver?



Sin tráfico, los intermediarios (NAT, firewall, etc.) cortan la conexión

La única forma de comprobar que el peer sigue ahí es renegociar TLS

Esto es caro



UDP no tiene sesión: no se sabe si el peer sigue ahí

La única forma de comprobarlo es renegociar DTLS

Esto es muy caro

Solución: HeartBEAT

Se añade un mensaje sencillo para mantener activa la conexión sin tener que renegociar

El nombre es una metáfora: mientras la conexión late, está viva

Bug: HeartBLEED

El nombre es otra metáfora: La conexión en vez de latir, sangra. Sangra memoria.



drovoh4k

Se publica el RFC 6520

Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) Heartbeat Extension

Abstract

This document describes the Heartbeat Extension for the Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) protocols.

The Heartbeat Extension provides a new protocol for TLS/DTLS allowing the usage of keep-alive functionality without performing a renegotiation and a basis for path MTU (PMTU) discovery for DTLS.

<https://www.rfc-editor.org/rfc/rfc6520.html>



drovoh4k

Cronología ● Marzo 2012

Se publica OpenSSL versión 1.0.1

OpenSSL 1.0.1 Branch Release notes

The major changes and known issues for the 1.0.1 branch of the OpenSSL toolkit are summarised below. The contents reflect the current state of the NEWS file inside the git repository.

Additional details of changes can be found in the [ChangeLog for OpenSSL 1.0.1](#).

A complete list of changes to OpenSSL 1.0.1 can be found in the [git repository commit log](#).

<https://slackware.cs.utah.edu/pub/openssl-web/news/openssl-1.0.1-notes.html>

⚠ Esta es la versión que se va a quedar dos años en producción, cifrando medio internet, con el bug dentro.



drovoh4k

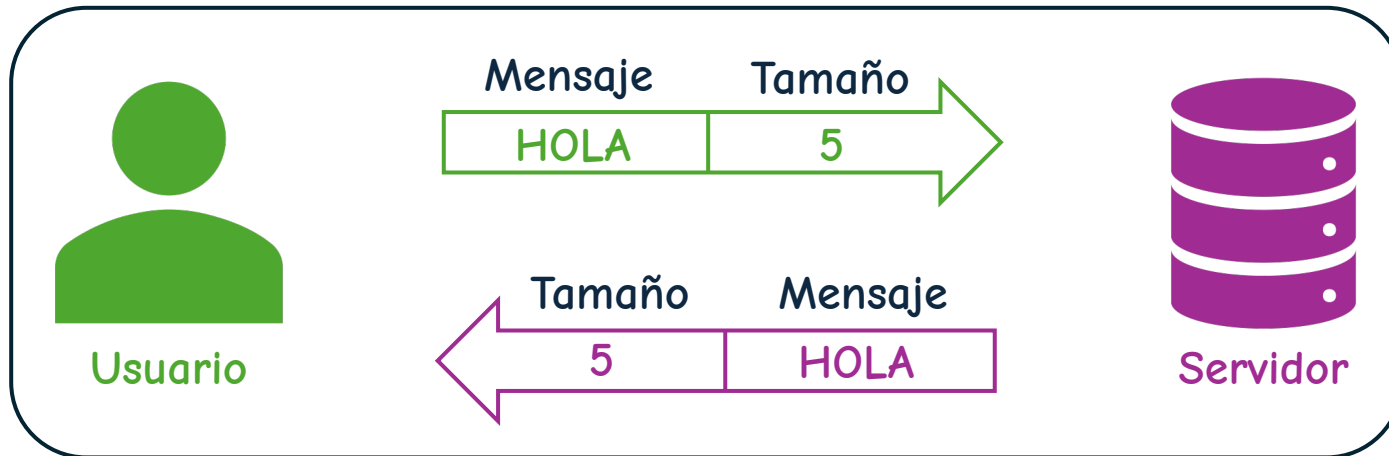
Entendiendo Heartbeat

Para entenderlo, debemos mirar el RFC, ya lo que hace OpenSSL es implementarlo tal y como se define ahí.

Definición simple

Heartbeat es una extensión de SSL, que define un protocolo donde:

- (1) El cliente manda un payload arbitrario
- (2) El servidor responde devolviendo exactamente el mismo payload



Y si vuelve igual, significa que la conexión esta viva



drovoh4k

Entendiendo Heartbeat

Estructura interna

```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[padding_length];  
} HeartbeatMessage;
```

type

Vale "1" si es request y vale "2" si es response

payload_length

Longitud del payload

payload

Bytes que el cliente pide que el servidor le devuelva

padding

Relleno aleatorio, requerido por TLS



drovoh4k

Entendiendo Heartbeat

Parte interesante

```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[padding_length];  
} HeartbeatMessage;
```

⚠ El cliente indica la longitud del payload que acaba de mandar

¿Para qué necesita el servidor que se lo digan?

El servidor ya recibió el paquete entero por TCP. Ya sabe cuántos bytes le han llegado.

¿Qué pasa si ese campo miente?



drovoh4k

El código vulnerable



drovoh4k

Overview

Descargar código vulnerable

openSSL
LIBRARY

Q

Roadmap

Community

Support

Sponsor

Policies

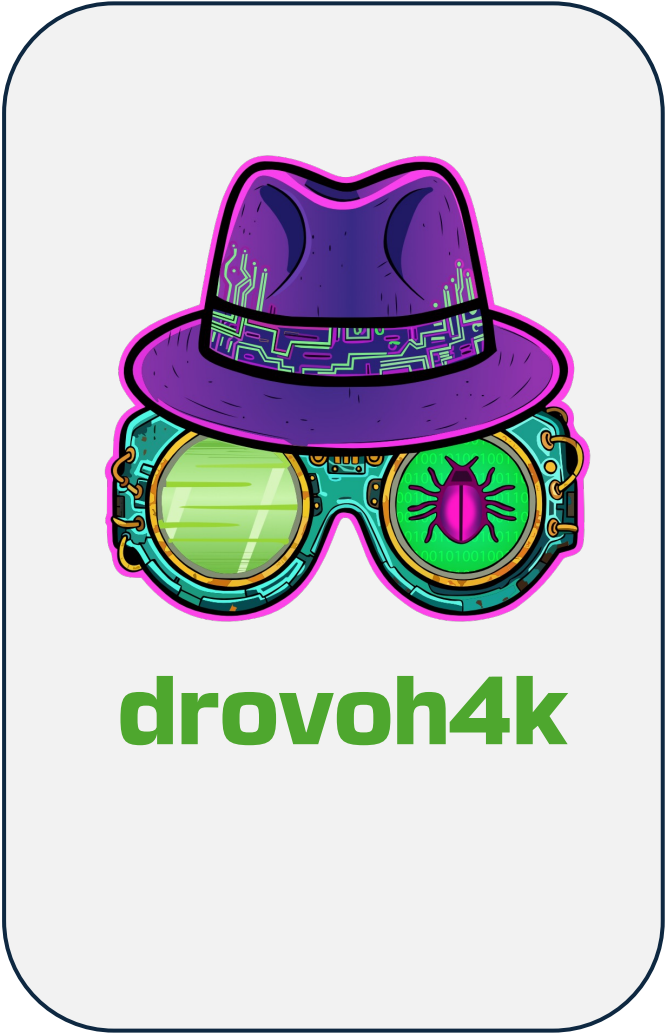
Downloads

Resources

About

Old 1.0.1 releases

Filename	Size	Release	Checksums
openssl-1.0.1u.tar.gz	4.4MiB	22 Sep 2016	(PGP) (SHA1) (SHA256)
openssl-1.0.1t.tar.gz	4.3MiB	03 May 2016	(PGP) (SHA1) (SHA256)
openssl-1.0.1s.tar.gz	4.3MiB	01 Mar 2016	(PGP) (SHA1) (SHA256)
openssl-1.0.1r.tar.gz	4.3MiB	28 Jan 2016	(PGP) (SHA1) (SHA256)
openssl-1.0.1q.tar.gz	4.3MiB	03 Dec 2015	(PGP) (SHA1) (SHA256)
openssl-1.0.1p.tar.gz	4.3MiB	09 Jul 2015	(PGP) (SHA1) (SHA256)
openssl-1.0.1o.tar.gz	4.3MiB	12 Jun 2015	(PGP) (SHA1) (SHA256)
openssl-1.0.1n.tar.gz	4.3MiB	11 Jun 2015	(PGP) (SHA1) (SHA256)
openssl-1.0.1m.tar.gz	4.3MiB	19 Mar 2015	(PGP) (SHA1) (SHA256)
openssl-1.0.1l.tar.gz	4.2MiB	15 Jan 2015	(PGP) (SHA1) (SHA256)
openssl-1.0.1k.tar.gz	4.2MiB	08 Jan 2015	(PGP) (SHA1) (SHA256)
openssl-1.0.1j.tar.gz	4.2MiB	15 Oct 2014	(PGP) (SHA1) (SHA256)
openssl-1.0.1i.tar.gz	4.2MiB	06 Aug 2014	(PGP) (SHA1) (SHA256)
openssl-1.0.1h.tar.gz	4.3MiB	05 Jun 2014	(PGP) (SHA1) (SHA256)
openssl-1.0.1g.tar.gz	4.3MiB	07 Apr 2014	(PGP) (SHA1) (SHA256)
openssl-1.0.1f.tar.gz	4.3MiB	06 Jan 2014	(PGP) (SHA1) (SHA256)



<https://openssl-library.org/source/old/1.0.1/>

Overview

Función clave

Esta es la función real que estuvo en producción durante dos años cifrando tráfico real, en millones de servidores:

```
└─ openssl-1.0.1f
  └─ ssl
    └─ t1_lib.c
      └─ f tls1_process_heartbeat
```

La vamos a analizar línea por línea con una sola pregunta en mente:

¿Dónde se valida que `payload_lenght` no miente?



drovoh4k

Análisis del código

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */

    /* Read type and payload length first */
    hbtype = *p++;
    n2s(p, payload);
    pl = p;

    if (s->msg_callback)
        s->msg_callback(0, s->version, TLS1_RT_HEARTBEAT,
            &s->s3->rrec.data[0], s->s3->rrec.length,
            s, s->msg_callback_arg);

    if (hbtype == TLS1_HB_REQUEST)
    {
        unsigned char *buffer, *bp;
        int r;

        /* Allocate memory for the response */
        buffer = OPENSSL_malloc(1 + 2 + payload + padding);
        bp = buffer;

        /* Enter response type, length and copy payload */
        *bp++ = TLS1_HB_RESPONSE;
        s2n(payload, bp);
        memcpy(bp, pl, payload);
        bp += payload;
        /* Random padding */
        RAND_pseudo_bytes(bp, padding);

        r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
    }
}
```



drovoh4k

Análisis del código

El puntero al paquete

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *p1;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */
```

unsigned char *p = &s->s3->rrec.data[0]

Apunta al primer byte del paquete que acaba de llegar, es decir, a lo que el atacante mandó.

```
struct {
    HeartbeatMessageType type;
    uint16 payload_length;
    opaque payload[HeartbeatMessage.payload_length];
    opaque padding[padding_length];
} HeartbeatMessage;
```



drovoh4k

Análisis del código

El puntero al paquete

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */
}
```

unsigned char *p = &s->s3->rrec.data[0]

Apunta al primer byte del paquete que acaba de llegar, es decir, a lo que el atacante mandó.

payload

Variable declarada, todavía vacía. La vigilarémos.

padding

Su valor no cambia en todo el análisis.



drovoh4k

Análisis del código

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */

    /* Read type and payload length first */
    hbtype = *p++;
    n2s(p, payload);
    pl = p;

    if (s->msg_callback)
        s->msg_callback(0, s->version, TLS1_RT_HEARTBEAT,
            &s->s3->rrec.data[0], s->s3->rrec.length,
            s, s->msg_callback_arg);

    if (hbtype == TLS1_HB_REQUEST)
    {
        unsigned char *buffer, *bp;
        int r;

        /* Allocate memory for the response */
        buffer = OPENSSL_malloc(1 + 2 + payload + padding);
        bp = buffer;

        /* Enter response type, length and copy payload */
        *bp++ = TLS1_HB_RESPONSE;
        s2n(payload, bp);
        memcpy(bp, pl, payload);
        bp += payload;
        /* Random padding */
        RAND_pseudo_bytes(bp, padding);

        r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
    }
}
```



drovoh4k

Análisis del código

La macro que mete la mentira en la variable

```
/* Read type and payload length first */  
hbtype = *p++;  
n2s(p, payload);  
pl = p;
```

n2s(p, payload)

Es una macro de OpenSSL, que significa “network to short”.

Lee 2 bytes desde donde apunta **p**, los interpreta en big-endian y los mete en la variable **payload**.

```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[padding_length];  
} HeartbeatMessage;
```



La variable se llama payload, pero contiene su longitud.



drovoh4k

Análisis del código

La macro que mete la mentira en la variable

```
/* Read type and payload length first */  
hbtype = *p++;  
n2s(p, payload);  
pl = p;
```

pl = p

Guardamos en **pl** la dirección donde empieza el payload real

Es decir, apunta a los bytes que el atacante mando como payload

```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[padding_length];  
} HeartbeatMessage;
```



drovoh4k

Análisis del código

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */

    /* Read type and payload length first */
    hbtype = *p++;
    n2s(p, payload);
    pl = p;

    if (s->msg_callback)
        s->msg_callback(0, s->version, TLS1_RT_HEARTBEAT,
            &s->s3->rrec.data[0], s->s3->rrec.length,
            s, s->msg_callback_arg);

    if (hbtype == TLS1_HB_REQUEST)
    {
        unsigned char *buffer, *bp;
        int r;

        /* Allocate memory for the response */
        buffer = OPENSSL_malloc(1 + 2 + payload + padding);
        bp = buffer;

        /* Enter response type, length and copy payload */
        *bp++ = TLS1_HB_RESPONSE;
        s2n(payload, bp);
        memcpy(bp, pl, payload);
        bp += payload;
        /* Random padding */
        RAND_pseudo_bytes(bp, padding);

        r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
    }
}
```



drovoh4k

Análisis del código

La validación no existe

```
if (s->msg_callback)
    s->msg_callback(0, s->version, TLS1_RT_HEARTBEAT,
        &s->s3->rrec.data[0], s->s3->rrec.length,
        s, s->msg_callback_arg);

if (hbtype == TLS1_HB_REQUEST)
{
```

if (s->msg_callback) ... S->msg_callback_arg);

Es un callback de debugging, que permite a OpenSSL registrar una función externa cada vez que pasa un mensaje, para fines de logging o trazas. ¡Pero no valida nada!

if (hbtype == TLS1_HB_REQUEST)

Entramos en el bloque de request, donde se construye la respuesta.

⚠ A partir de aquí empieza el bug



drovoh4k

Análisis del código

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */

    /* Read type and payload length first */
    hbtype = *p++;
    n2s(p, payload);
    pl = p;

    if (s->msg_callback)
        s->msg_callback(0, s->version, TLS1_RT_HEARTBEAT,
            &s->s3->rrec.data[0], s->s3->rrec.length,
            s, s->msg_callback_arg);

    if (hbtype == TLS1_HB_REQUEST)
    {
        unsigned char *buffer, *bp;
        int r;

        /* Allocate memory for the response */
        buffer = OPENSSL_malloc(1 + 2 + payload + padding);
        bp = buffer;

        /* Enter response type, length and copy payload */
        *bp++ = TLS1_HB_RESPONSE;
        s2n(payload, bp);
        memcpy(bp, pl, payload);
        bp += payload;
        /* Random padding */
        RAND_pseudo_bytes(bp, padding);

        r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
    }
}
```



drovoh4k

Análisis del código

El malloc que reserva lo que diga el atacante

```
if (hbtype == TLS1_HB_REQUEST)
{
    unsigned char *buffer, *bp;
    int r;

    /* Allocate memory for the response */
    buffer = OPENSSL_malloc(1 + 2 + payload + padding);
    bp = buffer;
```

buffer = OPENSSL_malloc(1 + 2 + payload + padding)

Se reserva memoria para la respuesta.

¿De qué tamaño?

1 byte + 2 bytes + **payload (definido por atacante)** + padding

```
struct {
    HeartbeatMessageType type;
    uint16 payload_length;
    opaque payload[HeartbeatMessage.payload_length];
    opaque padding[padding_length];
} HeartbeatMessage;
```

⚠ Si el atacante dice 3, se reservan 24 bytes y si dijo 65 535, se reservan ~64 KB.



drovoh4k

Análisis del código

El malloc que reserva lo que diga el atacante

```
if (hbtype == TLS1_HB_REQUEST)
{
    unsigned char *buffer, *bp;
    int r;

    /* Allocate memory for the response */
    buffer = OPENSSL_malloc(1 + 2 + payload + padding);
    bp = buffer;
```

bp = buffer

Apunta al inicio del buffer reservado, donde vamos a escribir la respuesta.



drovoh4k

Análisis del código

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */

    /* Read type and payload length first */
    hbtype = *p++;
    n2s(p, payload);
    pl = p;

    if (s->msg_callback)
        s->msg_callback(0, s->version, TLS1_RT_HEARTBEAT,
            &s->s3->rrec.data[0], s->s3->rrec.length,
            s, s->msg_callback_arg);

    if (hbtype == TLS1_HB_REQUEST)
    {
        unsigned char *buffer, *bp;
        int r;

        /* Allocate memory for the response */
        buffer = OPENSSL_malloc(1 + 2 + payload + padding);
        bp = buffer;

        /* Enter response type, length and copy payload */
        *bp++ = TLS1_HB_RESPONSE;
        s2n(payload, bp);
        memcpy(bp, pl, payload);
        bp += payload;

        /* Random padding */
        RAND_pseudo_bytes(bp, padding);

        r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
    }
}
```



drovoh4k

Análisis del código

El memcpy letal

```
/* Enter response type, length and copy payload */  
*bp++ = TLS1_HB_RESPONSE;  
s2n(payload, bp);  
memcpy(bp, pl, payload);  
bp += payload;
```

***bp++ = TLS1_HB_RESPONSE**

Escribimos el primer byte: el **type** de la respuesta

s2n(payload, bp)

La macro inversa de **n2s**, escribe los dos bytes del **payload_lenght** devuelto

```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[padding_length];  
} HeartbeatMessage;
```



drovoh4k

Análisis del código

El memcpy letal

```
/* Enter response type, length and copy payload */  
*bp++ = TLS1_HB_RESPONSE;  
s2n(payload, bp);  
memcpy(bp, pl, payload);  
bp += payload;
```

`memcpy(bp, pl, payload);`

Tenemos 3 argumentos:

`bp` --> Es el destino, la dirección donde escribimos la respuesta que le mandaremos al cliente

`pl` --> Es el origen, la dirección donde está el payload real que el atacante mandó dentro del paquete

`payload` --> Número de bytes a copiar.
Pero es el número que dijo el usuario,
no los bytes que realmente mandó.



drovoh4k

Análisis del código

El memcpy letal

```
/* Enter response type, length and copy payload */  
*bp++ = TLS1_HB_RESPONSE;  
s2n(payload, bp);  
memcpy(bp, pl, payload);  
bp += payload;
```

`memcpy(bp, pl, payload);`

Es decir, en un **ejemplo legítimo**, se vería así:

payload_lenght = 3
payload = ABC



payload_lenght = 3
payload = ABC

En cambio, un **ejemplo malicioso**, se vería así:

payload_lenght = 65 535
payload = NULL



payload_lenght = 65 535
payload = ????



drovoh4k

Análisis del código

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */

    /* Read type and payload length first */
    hbtype = *p++;
    n2s(p, payload);
    pl = p;

    if (s->msg_callback)
        s->msg_callback(0, s->version, TLS1_RT_HEARTBEAT,
            &s->s3->rrec.data[0], s->s3->rrec.length,
            s, s->msg_callback_arg);

    if (hbtype == TLS1_HB_REQUEST)
    {
        unsigned char *buffer, *bp;
        int r;

        /* Allocate memory for the response */
        buffer = OPENSSL_malloc(1 + 2 + payload + padding);
        bp = buffer;

        /* Enter response type, length and copy payload */
        *bp++ = TLS1_HB_RESPONSE;
        s2n(payload, bp);
        memcpy(bp, pl, payload);
        bp += payload;
        /* Random padding */
        RAND_pseudo_bytes(bp, padding);

        r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
```



drovoh4k

Análisis del código

Respuesta envenenada

```
r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
```

ssl3_write_bytes(...)

Es la función de OpenSSL que envía datos por el canal TLS.

La cantidad de datos que enviada depende del valor indicado por el usuario,



drovoh4k

El parche

En [OpenSSL 1.0.1g](#), publicada el 7 de abril de 2014, añadieron esto antes del [OPENSSL_malloc](#):

```
int tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */

    /* Read type and payload length first */
    hbtype = *p++;
    n2s(p, payload);
```

```
if (1 + 2 + payload + 16 > s->s3->rrec.length)
    return 0; /* silently discard per RFC 6520 sec. 4 */
```

```
if (hbtype == TLS1_HB_REQUEST)
{
    unsigned char *buffer, *bp;
    int r;

    /* Allocate memory for the response */
    buffer = OPENSSL_malloc(1 + 2 + payload + padding);
    bp = buffer;

    /* Enter response type, length and copy payload */
    *bp++ = TLS1_HB_RESPONSE;
    s2n(payload, bp);
    memcpy(bp, pl, payload);
    bp += payload;
    /* Random padding */
    RAND_pseudo_bytes(bp, padding);

    r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
```



drovoh4k

Qué hay en esa memoria

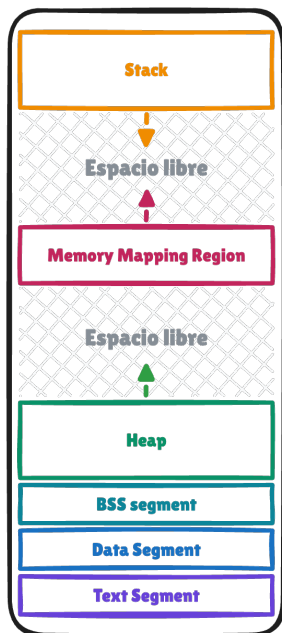


drovoh4k

Overview

En Linux, [OpenSSL](#) se distribuye como una librería compartida.

Cuando un proceso como [nginx](#), [apache](#) o [postfix](#) arranca, el linker dinámico la carga dentro del espacio de direcciones virtuales de ese proceso.



A partir de ese momento, el código de OpenSSL y el código del server **ocupan secciones distintas del mismo proceso, pero comparten el mismo heap.**

Y [OpenSSL](#) reserva espacios memoria, de los que posteriormente podemos filtrar ~64 KB de memoria adyacente.



drovoh4k

Datos Filtrados

El heap de un servidor en producción contiene cosas como:

- La clave privada del certificado

Necesaria para que el servidor negocie conexiones TLS.

```
-----BEGIN RSA PRIVATE KEY-----
MIIIEowIBAACAQEAqwitjHs181z19cGdvgne1ck8IYQUPyKAPtkPU3HZ+FwY3aTw
1IizXTH1QHxAMrpYF8MACK/1zC4aV1aUW6DxcXWYQnDkddJ8DKYgKxqE0m3Adtyy
XknNH1xwG7EDWXL5aysJ+kubtDqqhOHuMCPNbCNJ+zcnpsetccdnwL2prISFLU05
LrLZ/NqHxXKPeP5F6GrxRZj1DTMeQ7TUG+MawJG0vxN2GHdyU+CBx7nvQ2xI0nCL
mkVSFBsw1vrP1LPEVi7Dwh8oD5nNqslxitDBI+nTZbmX9iCFFjxfeGh6VryVRaGK
/31FH21rNMVh6Kk2HfK3j5MJZ5dft3gbPznZiQIDAQABoIBABtnn49D6sfz10FS
jbj2kesFfIoORMQ8/fmUBB21v3xJHX7eP+x+HudO6vIyK+K08wqTpzz/w1luAbBV
3R21cxpv9mko96JdYJ0SPVnKw3+IMG+b00VaY/CjdfSn71Z00rw2QDohXwIiIFIU
dXrnMbIjEWacVsB7dIn8jBw82/95N4n+9rh1oNccH9jeCoRIgsLVbpu8tcHsSFN
Uk5SwNRfPYUoq7Q6jD6HNbeEtabyvJfumDbdiB8DusUW83RohKpNaP8+z1ZAMBUp
IySayM4ZNn79ZwGBFZXpe2Ib4MII8IYtxH0vd3zzMRfXf8nsWjuXCFueOVY93Xgn
+ejgYhUCgYEA0n0GvU7LxgIdlc6zi5BULV0ryBsWrQ2NRC+1HG4M1woJpZs/VFTY
3XQN8qNaz9VvLdo6GVP0hIfFUVs6JA+MdqLMC6+i0cOfdMNs1APOCtQHXRSI0s+h
XrNFXHd0OyqG373Q9JNfcdLHkaSGWpaQqX3AgJTny7Dr8VJ801ym1MCgYEA0APD
VZEf3uJP1HIN16FqGjKGDzVdcv1W520ViKMPp39vH1eOMWsu1J9g8uG1wqDn+vGy
8cm/PQrv1+CvFGmhdoop34Z1pKXyTagUIV1TqeG90xsLnwlvZi9//PGE67pxLejJ
L0DH+L3DoAMUECsG8LSt942gC3OfjGbFwAYBGTMCgYEAxnhTOX8pzwwBn7bmPuuZ
I6kd1jJMoGIN2MRvL15qCqTRUdDf1wG19rHuU0JMx3mFuqNxcT5hyfFHUyZFvSjY
FikM4tv6V3XwTaXTYNoxlq40QM1T0gsrKmM5PKX596vyvtd5RpQLQ5m0DFkYe1R3
xS4+iStuEbNLmtoZDUrh37cCgYBiyna5q6vcOdU6C1ThpOp0OKyMioawNLnOePAq
rW6g2RfYZcfFU5DosCzmDSY6ohqqhqG6zVsCza/s8euUoToqxxUUpzIOuTVjhkDR
OdbPLUPu/7g8NsnuWP7LDZT1ak2YRSdPRs/yJYXD99JOf3y8X2cD3AI3jhknICQo
t3FMGwKBgDR7+ya7Tn8KZd19oANZjuqIu9DF1iTeIQc7fRcV1UDJxQaT7jowwSM
Eex1tK3KQc27rK1j77ED3maG9L19RP5SuFQVsK78pBosMNZxNfDp4wCtsjWicYcG
OG1Zr1QgVgowerDjPYa9gLomCum+PNKNA7fTxqVqb8Z6957kTgFn
-----END RSA PRIVATE KEY-----
```



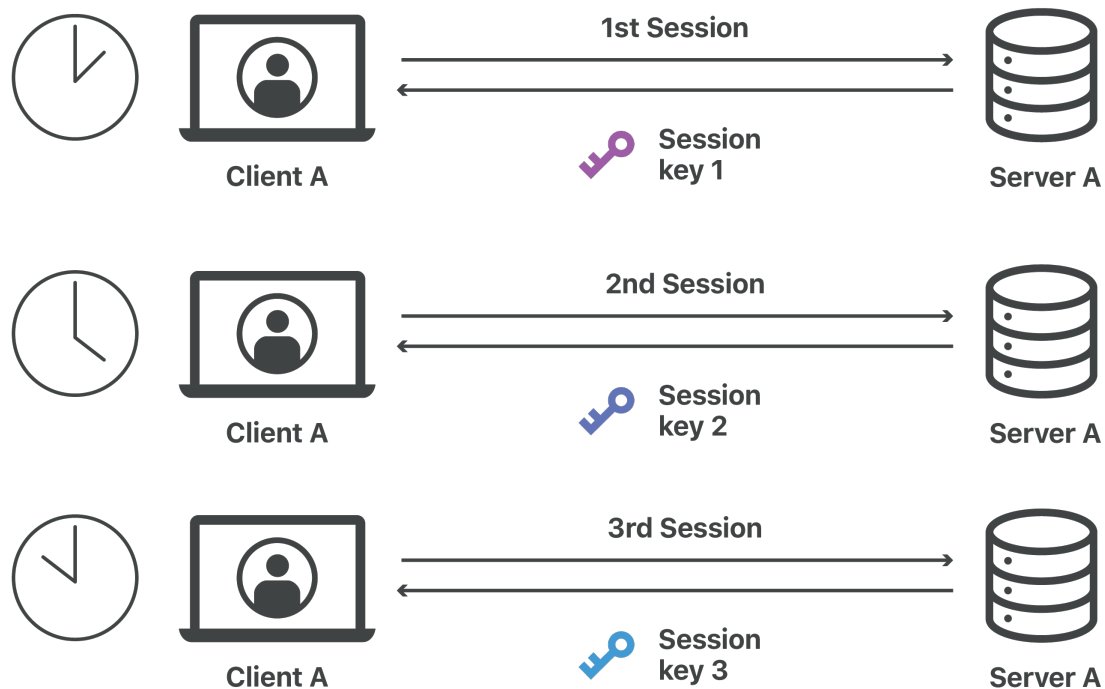
drovoh4k

Datos Filtrados

El heap de un servidor en producción contiene cosas como:

- Las session keys de todas las sesiones TLS activas en ese momento

Permiten cifrar y descifrar el tráfico de usuarios conectados en ese momento.



drovoh4k

Datos Filtrados

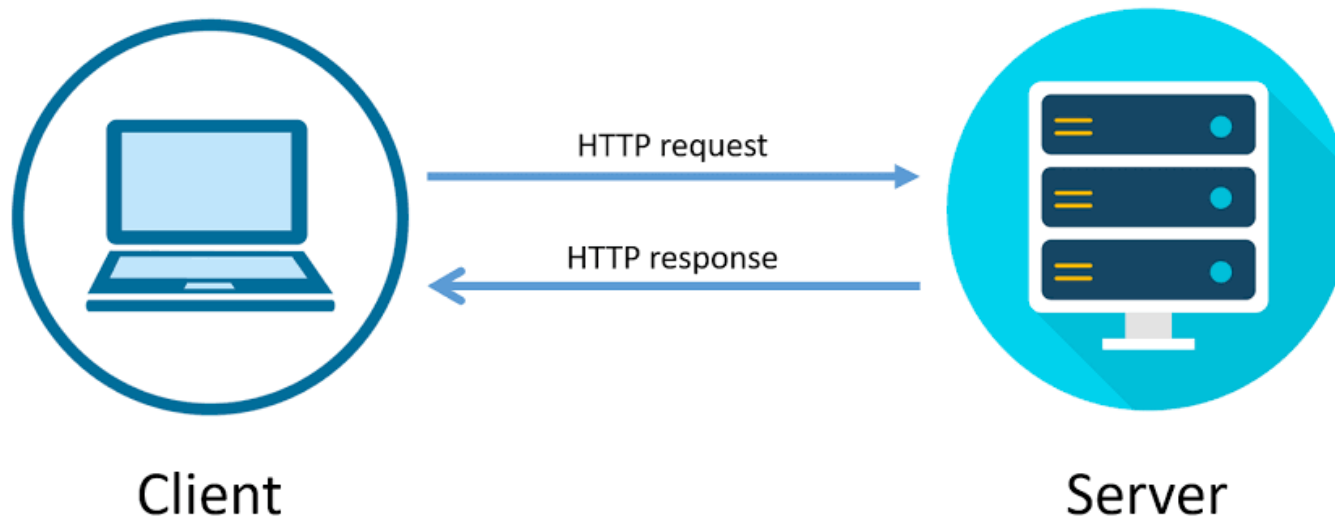
El heap de un servidor en producción contiene cosas como:

- **Datos en curso de otras peticiones HTTPS:**

Cabeceras HTTP, incluyendo `Cookie`, `Authorization`, etc.

Cabeceras POST, con formularios de login, tarjetas de pago, etc.

Tokens de sesión, JWTs, refresh tokens.



drovoh4k

Puntos clave

- **El heap cambia constantemente**

La memoria se reutiliza en cada petición HTTPS, así que cada heartbeat puede devolver un fragmento distinto.

- **El ataque es probabilístico**

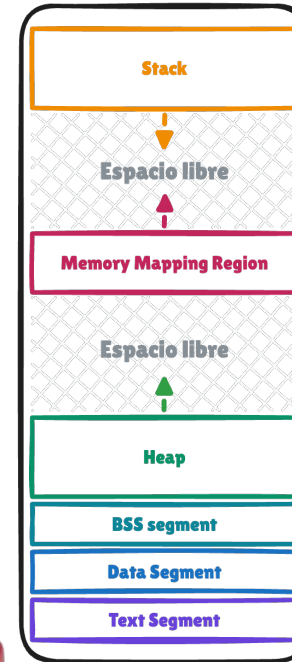
Repitiendo muchos heartbeats, el atacante aumenta la probabilidad de capturar datos sensibles.

- **Es difícil de detectar y escalar**

Heartbeat va por debajo de HTTP no aparece en los logs del servidor.

Además, no por defecto no hay rate limit,

Un atacante puede mandar miles de heartbeats por segundo. Filtrando 64KB del heap en cada uno.



drovoh4k

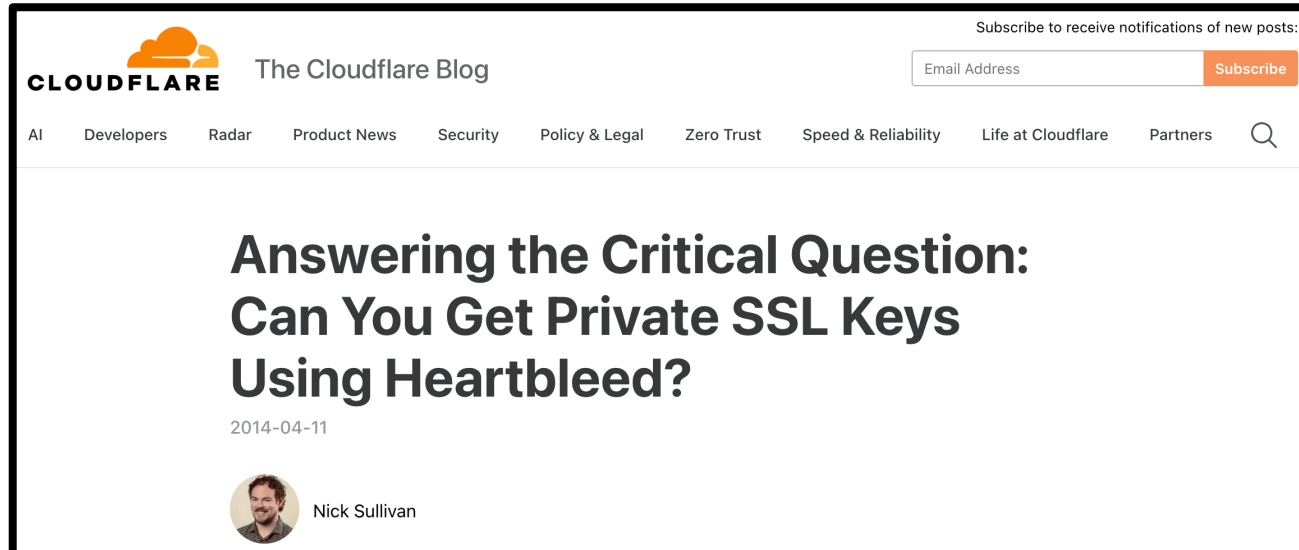
La gran pregunta

A nivel teórico esta claro, pero a nivel práctico:

¿Es posible sacar la clave privada del servidor?

Si esto es posible, habría que rotar TODOS los certificados TLS del planeta.

Cloudflare decidió zanjar el debate, montó un servidor público con su clave privada cargada y retó al mundo.



drovoh4k

<https://blog.cloudflare.com/answering-the-critical-question-can-you-get-private-ssl-keys-using-heartbleed/>

La gran pregunta

Captura del
11 de Abril
del 2014

 CLOUDFLARE

The Heartbleed Challenge

Can you steal the keys from this server?

Has the challenge been solved yet? **NO.**

This server is running nginx-1.5.13 linked against OpenSSL 1.0.1.f on Ubuntu 13.10 x86_64. It is vulnerable to [Heartbleed](#). Can you get the secret key?

If you think you have it, submit the RSA signature of the string "Proof I have your key\n" as proof. This proof can be obtained with the following commands:

```
echo "Proof I have your key" | openssl sha1 -sign server.key -sha1 | openssl enc -base64
```

Signature

Email Address

Email Address

Submit



drovoh4k

<https://web.archive.org/web/20140411171811/>
<https://www.cloudflarechallenge.com/heartbleed>

La gran pregunta

Captura del
12 de Abril
del 2014

 CLOUDFLARE

The Heartbleed Challenge

Can you steal the keys from this server?

Has the challenge been solved yet? **YES**

Can you see this site? You shouldn't be able to, we have revoked the certificate. If you can still see this message, Certificate Revocation may be broken in your browser. See this [post](#) for more details.

So far, at least four people have independently solved the Heartbleed Challenge.

The first was submitted at 4:22:01PST by Fedor Indutny ([@indutny](#)). He sent at least 2.5 million requests over the span of the challenge, this was approximately 30% of all the requests we saw. The second was submitted at 5:12:19PST by Ilkka Mattila of NCSC-FI using around 100 thousand requests.

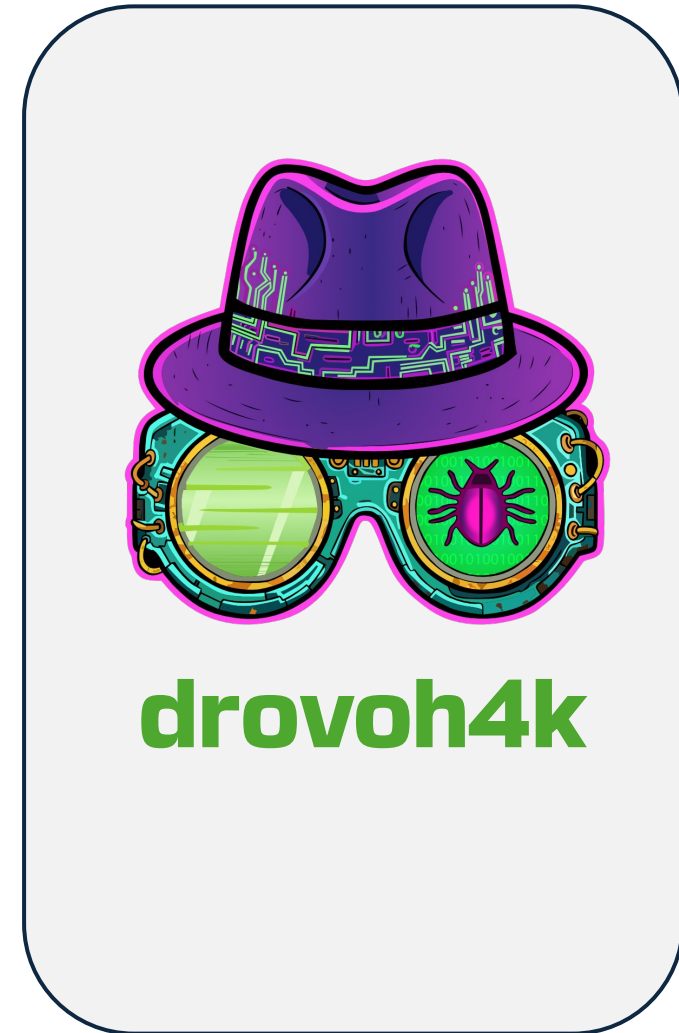
UPDATE: Two more confirmed winners: Rubin Xu, PhD student in the Security group of Cambridge University submitted at 04:11:09PST on 04/12; and Ben Murphy ([@benmmurphy](#)), Security Researcher submitted at 7:28:50PST on 04/12.

We confirmed that both of these individuals have the private key and that it was obtained through Heartbleed exploits. We rebooted the server at 3:08PST, which may have contributed to the key being available in memory, but we can't be certain.

This server is running nginx-1.5.13 linked against OpenSSL 1.0.1.f on Ubuntu 13.10 x86_64. It is vulnerable to [Heartbleed](#). Can you get the secret key?

If you think you have it, submit the RSA signature of the string "Proof I have your key\n" as proof. This proof can be obtained with the following commands:

```
echo "Proof I have your key" | openssl sha1 -sign server.key -sha1 | openssl enc -base64
```



<https://web.archive.org/web/20140412234126/>
<https://www.cloudflarechallenge.com/heartbleed>

Demo



drovoh4k



Tu pedido, conectado de principio a fin.

GloByte simula una app de delivery: eliges productos, envías tus datos al servidor y recibes actualizaciones de tracking en tiempo real.

Esta demo incluye una simulación educativa de Heartbleed. No usa OpenSSL real.

[Ver WriteUp técnico](#)

MENÚ

Elige productos

Byte Burger

Burger con cheddar y salsa amarilla.

9.90 €

Añadir

Data Wrap

Wrap crujiente con vegetales y lima.

7.50 €

Añadir

Pixel Bowl

Bowl completo con arroz, pollo y salsa verde.

8.80 €

Añadir

CHECKOUT

Datos y resumen

Nombre

Drovoh4k

Dirección

Avenida Byte 404

Aún no has añadido productos.

Total

0.00 €

Finalizar pedido



drovoh4k

drovoh4k

DrovoLeaks

Public

Unpin

Watch 1

Fork 2

Star 28

main

1 Branch

0 Tags

Go to file

Add file

<>

 drovoh4k	Nueva clase: Metodología análisis de binarios	6a7d86c · 2 weeks ago	🕒 42 Commits
📁 cursos	Nueva clase: Metodología análisis de binarios	2 weeks ago	
📁 masterclass	Refactor: meta.json removed + Links corrected + Images c...	2 months ago	
📁 tutoriales	Refactor: meta.json removed + Links corrected + Images c...	2 months ago	
📁 varios/ida_vs_ghidra	Nuevo video: Ida VS Ghidra	4 months ago	
📁 vulnerabilidades	Nueva vulnerabilidad: CVE-2026-3909 y CVE-2026-3910 ...	last month	
📁 writeups	Nuevo CTF: Hack4uCTF (crypto) - Locked Away	last month	
📄 README.md	Nueva masterclass: Introducción a MIPS	3 months ago	

📖 README

DrovoLeaks

📺 Channel @drovoh4k

Este repositorio es el **material de apoyo** de mis vídeos de YouTube.

La idea es tener todo el conocimiento del canal en un lugar centralizado y ordenado, no solo los propios videos, sino también todos los recursos extra.

Navegación rápida

- 📄 [Writeups](#)
 - Resoluciones de retos (crackmes, plataformas y CTFs), con scripts y recursos.
- 📺 [Cursos](#)
 - Contenido estructurado en módulos y clases: enfoque progresivo, con enlaces a playlist y recursos.
- 🎓 [Masterclass](#)

About

No description, we provided.

📖 Readme

📊 Activity

☆ 28 stars

👁 1 watching

🍴 2 forks

Releases

No releases published
[Create a new release](#)

Contributors 1

 drovoh4k Llo

Languages

- Python 76.4%
- C 6.2%
- Doc 3.5%
- Shell 3.5%

Suggested workflows

Based on your tech

 Publish Python Package

Publish a Python Package on release.

 Python application

Create and test a Python application.

drovoh4k



SUSCRÍBETE



Drovohub



drovoh4k